

A3-PS023-TC061

Minimal Two-Node Cloud Hypervisor VM Controller

Abid Shaikh & Umar Ansari
abid.231446.it@mhssce.ac.in | umar.231405.it@mhssce.ac.in

AIORI Hackathon 3
October 5, 2026

Problem Description

Managing virtual machines across multiple KVM-enabled Linux hosts requires coordinating VM lifecycle operations, host resources, and runtime state. Without a centralized controller, these tasks can become fragmented, error-prone, and difficult to automate reliably.

- **Fragmented management:** VM creation, startup, shutdown, and status checks must be coordinated across separate hosts.
- **Inefficient resource allocation:** A host may receive a VM despite another host having more available memory.
- **Resource overcommitment:** Concurrent VM requests can consume the same apparent free capacity unless memory reservations are coordinated.
- **Inconsistent VM state:** Controller metadata can disagree with the actual Cloud Hypervisor process state after timeouts, crashes, or host failures.
- **Unreliable retries:** Retrying a timed-out request without checking the original operation can create duplicate work or conflicting state.
- **Operational complexity:** Manual checks and cleanup make repeatable deployments and troubleshooting harder.
- **Security risks:** Exposed management interfaces, unsafe image paths, or insufficient authentication can allow unauthorized VM operations.

Desired outcome: A lightweight, secure, and reliable controller that manages VM lifecycle operations across two hosts and places VMs according to available memory.

Background

Why optimize this setup?

A two-node environment is small enough to keep the control plane lightweight, but still requires coordination across hosts. The main optimization opportunity is to combine **memory-aware placement, atomic resource reservations, safe lifecycle handling,** and **reconciliation** without introducing the operational overhead of a large cluster-management platform.

KVM — Kernel-based Virtual Machine

KVM provides **Linux kernel virtualization capabilities** using **hardware-assisted CPU virtualization**. It allows Linux hosts to run virtual machines, provided the hardware, kernel, and permissions support it.

Cloud Hypervisor

Cloud Hypervisor is a **lightweight virtual machine monitor (VMM)** designed for modern cloud workloads. It uses **KVM for virtualization** and exposes a **management API** for VM configuration and lifecycle operations.

Centralized VM orchestration

A **controller** receives user requests, maintains VM metadata, selects a host, and delegates execution to an **agent on that host**. The agent interacts with the **local Cloud Hypervisor instance.**

Literature & RFC Review

The proposed controller does not need to invent a new virtualization protocol. It combines existing technologies with a small coordination layer tailored to a two-host environment.

Reference	Relevant finding	Application to our solution
Cloud Hypervisor API documentation	Provides REST-based VM and VMM operations, including API access through a local Unix socket.	The worker agent uses the local management API rather than exposing Cloud Hypervisor directly to clients.
Linux KVM API documentation	Defines the kernel interfaces used to create and manage VMs, virtual CPUs, and related resources.	Establishes the host prerequisites and the virtualization layer beneath Cloud Hypervisor.
RFC 9110 - HTTP	Defines idempotent HTTP methods and cautions against automatically retrying non-idempotent requests without a safe mechanism.	Supports explicit retry semantics, operation identifiers, and idempotency handling for lifecycle requests.

Reference	Relevant finding	Application to our solution
SQLite Atomic Commit documentation	Explains how transactions provide all-or-nothing database updates.	Enables atomic placement reservations and durable operation-state updates in the controller database.
OWASP REST Security Cheat Sheet	Recommends access control, strict input validation, method restrictions, and appropriate request handling.	Informs API authentication, validation of VM specifications, and protection of worker endpoints.

Proposed Solution

1 Central controller

Exposes authenticated APIs for VM creation, starting, stopping, and listing. It validates requests, records operations, evaluates host capactiy, and selects a worker.

2 Persistent metadata and reservation store

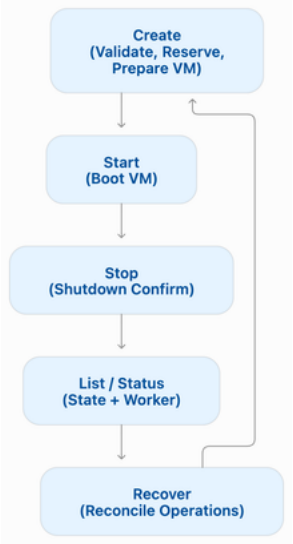
Stores VM definitions, assigned hosts, desired states, operation status, and memory reservations. Database transactions prevent concurrent requests from claiming the same available capacity.

3 Worker agents on Host A and Host B

Report host capacity, validate local prerequisites, manage Cloud Hypervisor processes, and return execution results. The VMM management socket remains local to the host.

4 Reconciliation and recovery

Compares the controller's recorded state with worker observations and actual VMM processes. Uncertain operations are checked before retrying or releasing reservations.



Optimization Proposed

From Manual VM Operations to Resource-Aware, Reliable Orchestration

01 Key improvements

Before Manual commands, Fixed placement, Uncertain retries, Manual recovery

After

Unified API — both hosts
Smart placement — memory-aware
Concurrency — atomic reservations
Reliability — idempotency and lifecycle tracking
Recovery & security — reconciliation, restricted socket
Observability — structured logs and status

02 Memory-aware placement

$$C_h = \max(0, \min(A_h - S_h, L_h - R_h))$$

Select the eligible host with the greatest safe capacity; reserve memory atomically before dispatch.

Example · Request: 4 GiB

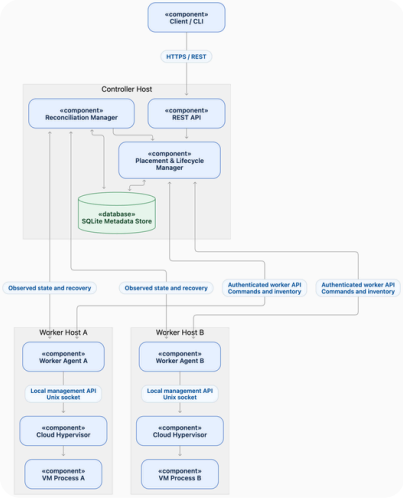


Illustrative values, not benchmark results.

03 Success metrics to validate

- Cross-host lifecycle success
- Correct concurrent placement
- No duplicates or leaked reservations
- Restart recovery and request latency

Solution Architecture & Design



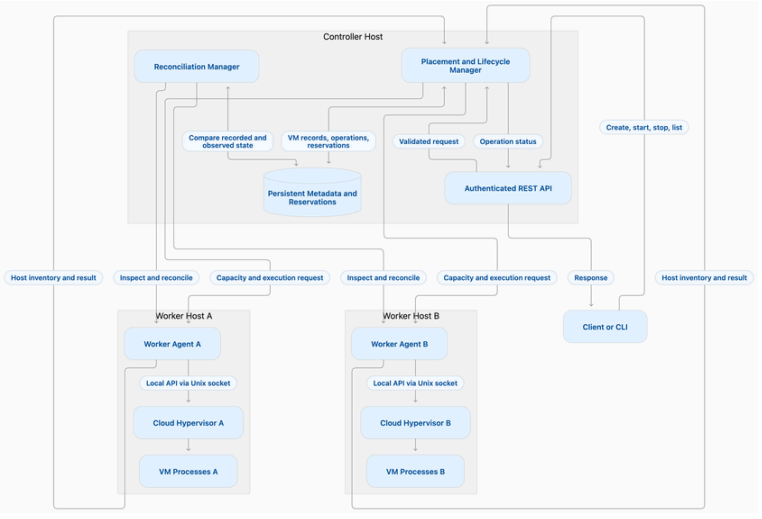
- **Client / CLI:** Sends VM creation, start, stop, and listing requests.
- **REST API:** Authenticates requests and validates their inputs.
- **Placement & Lifecycle Manager:** Selects a host based on available memory, manages reservations, and coordinates VM operations.
- **SQLite Metadata Store:** Persists VM definitions, host assignments, reservations, and operation states.
- **Reconciliation Manager:** Compares recorded state with worker observations and handles uncertain operations.
- **Worker Agents:** Execute commands and report host inventory and VM status.
- **Cloud Hypervisor:** Creates and runs VMs through the local KVM virtualization interface.
- **VM Processes:** The actual virtual machine instances running on each host.

Data Flow Diagram

This diagram complements the earlier architecture design by showing the control path, metadata path, resource feedback, and local VM execution path.

Main data flows:

1. Request flow
2. Decision flow
3. Persistence flow
4. Execution flow
5. Feedback flow
6. Recovery flow



Deployment Timeline

Dates	Milestone	Activities	Deliverable
16-17 Oct	**M1 - Environment validation**	Verify both Linux hosts, KVM access, Cloud Hypervisor version/API, networking, guest image, and memory reporting.	A test VM boots and shuts down on both hosts; process and socket cleanup verified.
18-20 Oct	**M2 - Worker implementation**	Implement local VM create/start/stop/status operations, input validation, process supervision, and worker authentication.	A worker can manage a VM reliably on one host.
21-23 Oct	**M3 - Controller and persistence**	Implement client API, VM metadata, operation records, idempotency handling, and transactional reservations.	Controller and one worker complete the core lifecycle with durable state.
24-26 Oct	**M4 - Two-node placement**	Register both workers, collect capacity, implement placement selection, and connect controller dispatch to both hosts.	VM requests are routed to the appropriate eligible host.
27-29 Oct	**M5 - Reliability and security**	Test concurrent requests, timeouts, restarts, uncertain outcomes, reconciliation, API authentication, and resource cleanup.	No duplicate VM execution or reservation leaks in the defined test scenarios; remaining limitations documented.
30-31 Oct	**M6 - Final deployment and submission**	Run end-to-end demonstrations, capture logs and metrics, fix critical defects, and prepare slides, diagrams, and final documentation.	Repeatable two-node demo, evidence of test results, and final submission package.

Bibliography I

[1] **Cloud Hypervisor Project.** *Cloud Hypervisor API Documentation*. Official project documentation describing the REST API, VM lifecycle endpoints, and Unix-socket interface. [Link]

[2] **Linux Kernel Documentation.** *The Definitive KVM API Documentation*. Official documentation for the kernel virtualization API, VM creation, virtual CPUs, and device interfaces. [Link]

[3] **Fielding, R., Nottingham, M., and Reschke, J.** *RFC 9110: HTTP Semantics*. Internet Engineering Task Force (IETF), 2022. Defines HTTP semantics, including idempotent methods and safe retry considerations. [Link]

[4] **SQLite Project.** *Atomic Commit In SQLite*. Official technical explanation of transaction atomicity and crash-safe database updates. [Link]

[5] **OWASP Foundation.** *REST Security Cheat Sheet*. Security guidance for authentication, access control, input validation, HTTP methods, and API request handling. [Link]

Thank You for Your Attention!